Em programação, **estado** é a condição ou o conjunto de dados (valores de variáveis, objetos ou registradores) que um sistema, programa ou componente guarda **em um determinado momento**. [1, 2, 3, 4]

Em termos simples, é a **"fotografia" atual da memória** que diz o que o programa está fazendo ou o que aconteceu até agora. [1, 2]

Principais conceitos relacionados ao Estado

- **Estado de um objeto ou variável:** É o valor que ele armazena agora. Por exemplo, em um botão de interface, o estado dele pode ser `ativado` ou `desativado`; em um carrinho de compras, o estado é a lista de itens que o usuário adicionou.
- **Sistemas com Estado (*Stateful*) vs. Sem Estado (*Stateless*):**
  - **Stateful:** O sistema lembra de interações passadas. Uma sessão de login em um site é *stateful* porque o servidor lembra que você já fez login.
  - **Stateless:** O sistema não guarda nada entre uma requisição e outra. Cada pedido é isolado, como uma consulta simples na internet em que o servidor apenas responde e esquece quem você é. [1]
- **Máquina de Estados (*State Machine*):** Um modelo de projeto muito usado em jogos e lógica de controle, onde um objeto só pode estar em um **estado finito** por vez (ex: um personagem de jogo que pode estar em `parado`, `andando` ou `atacando`) e muda de estado através de **eventos** específicos. Você pode entender melhor como isso funciona no artigo do Refactoring.Guru sobre o padrão State. [1, 2, 3]
- **Padrão de Projeto State:** É uma técnica da Orientação a Objetos onde o comportamento de um objeto muda dinamicamente quando o seu estado interno é alterado, fazendo parecer até que ele virou outra classe. [1]